

JENKINS CHALLENGE

Use the below code and implement our CICD stages.

- **Source Code:** <https://github.com/rajkumari-hub/hiring-app>
- Repository contains the application code for the hiring app.
- Create one Declarative pipeline job
- Create one Scripted pipeline job
- Create one multi stage pipeline job
- Create one parallel stage pipeline job

Jenkins officially supports both **Declarative** and **Scripted** Pipeline syntax, and Declarative Pipeline also supports sequential and parallel stages.

1. What you need to create

# Jenkins Job	Purpose
1 Hiring-App-Declarative	Declarative Pipeline
2 Hiring-App-Scripted	Scripted Pipeline
3 Hiring-App-Multi-Stage	Multiple sequential stages
4 Hiring-App-Parallel	Parallel execution of stages

For your CI/CD flow, we'll use:

Git Clone → Build → Test → Package

For example:

GitHub



Git Clone



Build



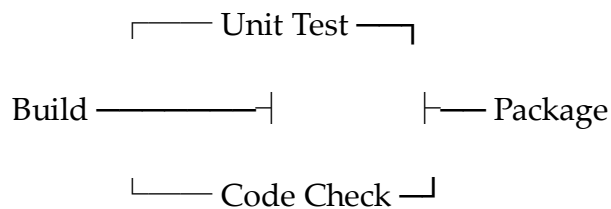
Test



Package

In the **multi-stage** job, these stages execute one after another.

In the **parallel** job:



Part 1 — Before creating the jobs

Step 1: Verify Jenkins tools

On your Jenkins server, check:

```
java -version
```

```
git --version
```

```
mvn -version
```

You should have:

Java

Git

Maven

You already installed Java, Git and Maven during your previous Jenkins setup.

Step 2: Verify the application manually

On the Jenkins server:

```
cd /tmp
```

```
git clone https://github.com/betawins/hiring-app.git
```

```
cd hiring-app
```

Now check the files:

```
ls -la
```

Look for:

```
pom.xml
```

```
src/
```

Then run:

```
mvn clean test
```

If that succeeds, run:

```
mvn clean package
```

Then:

```
ls target/
```

You should see the generated application artifact, such as a .war or .jar, depending on the project configuration.

```
us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?ad...
aws [United States (N. Virginia) v] Rajkumari (571833381790) Rajkumari

[root@ip-172-31-20-75 tmp]# ls -la
total 0
drwxrwxrwt. 14 root    root    280 Aug 17 13:31 .
dr-xr-xr-x. 18 root    root    253 Aug 14 13:07 ..
drwxrwxrwt. 2 root    root     40 Aug 17 13:26 .ICE-unix
drwxrwxrwt. 2 root    root     40 Aug 17 13:26 .X11-unix
drwxrwxrwt. 2 root    root     40 Aug 17 13:26 .XIM-unix
drwxrwxrwt. 2 root    root     40 Aug 17 13:26 .font-unix
drwxr-xr-x. 4 root    root    220 Aug 17 13:31 hiring-app
drwxr-xr-x. 2 jenkins jenkins 60 Aug 17 13:26 hspcrfdata_jenkins
drwxr-xr-x. 2 root    root     40 Aug 17 13:27 hspcrfdata_root
drwx-----. 3 root    root     60 Aug 17 13:26 systemd-private-cad9b7fa9e0242b38f51c95450669b
b8-chrond.service-oi705q
drwx-----. 3 root    root     60 Aug 17 13:26 systemd-private-cad9b7fa9e0242b38f51c95450669b
b8-dbus-broker.service-xl9ioj
drwx-----. 3 root    root     60 Aug 17 13:26 systemd-private-cad9b7fa9e0242b38f51c95450669b
b8-policy-routes@ens5.service-9uHOCH
drwx-----. 3 root    root     60 Aug 17 13:26 systemd-private-cad9b7fa9e0242b38f51c95450669b
b8-systemd-logind.service-CdgfBZ
drwx-----. 3 root    root     60 Aug 17 13:26 systemd-private-cad9b7fa9e0242b38f51c95450669b
b8-systemd-resolved.service-cHuHMv
[root@ip-172-31-20-75 tmp]# cd hiring-app
[root@ip-172-31-20-75 hiring-app]# ls
Dockerfile  README.md      jenkinsfile-cicd  src
Jenkinsfile 'Untitled Diagram.drawio' pom.xml
[root@ip-172-31-20-75 hiring-app]#
```

i-0048f2c1d99b49893 (Jenkins-Server)

PublicIPs: 54.172.207.1 PrivateIPs: 172.31.20.75



```
us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?ad...
aws United States (N. Virginia) Rajkumari (571833381790) Rajkumari
root@ip-172-31-20-75 hiring-app]# mvn clean test
[INFO] Scanning for projects...
[INFO] -----< in.javahome:hiring >-----
[INFO] Building hiring Maven Webapp 0.1
[INFO] -----[ war ]-----
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ hiring ---
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ hiring ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is p
atform dependent!
[INFO] skip non existing resourceDirectory /tmp/hiring-app/src/main/resources
[INFO] --- maven-compiler-plugin:3.1:compile (default-compile) @ hiring ---
[INFO] No sources to compile
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ hiring ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is p
atform dependent!
[INFO] skip non existing resourceDirectory /tmp/hiring-app/src/test/resources
[INFO] --- maven-compiler-plugin:3.1:testCompile (default-testCompile) @ hiring ---
[INFO] No sources to compile
[INFO] --- maven-surefire-plugin:2.12.4:test (default-test) @ hiring ---
[INFO] No tests to run.
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.527 s
[INFO] Finished at: 2026-08-17T13:38:59Z
[INFO] -----
root@ip-172-31-20-75 hiring-app]#
```

i-0048f2c1d99b49893 (Jenkins-Server) ×

PublicIPs: 54.172.207.1 PrivateIPs: 172.31.20.75

Part 2 — Create Declarative Pipeline Job

This is your **first assignment**.

Step 1: Create the job

Go to Jenkins:

Dashboard



New Item

Enter:

Hiring-App-Declarative

Select:

Pipeline

Click:

OK

← → ↻ ⚠ Not secure 54.172.207.1:8080/view/all/newJob ☆ ⚙

Jenkins / All ▾ / New Item 🔍 ⚙ 👤

New Item

Enter an item name

Hiring-App-Declarative

Select an item type

- Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
- Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

OK

Step 2: Go to Pipeline section

Scroll down to:

Pipeline

Select:

Definition:

Pipeline script

```
pipeline {
  agent any

  stages {

    stage('Git Clone') {
      steps {
        git branch: 'main',
            url: 'https://github.com/betawins/hiring-app.git'
      }
    }

    stage('Build') {
      steps {
        sh 'mvn clean compile'
      }
    }

    stage('Test') {
      steps {
        sh 'mvn test'
      }
    }

    stage('Package') {
      steps {
        sh 'mvn package -DskipTests'
      }
    }
  }

  post {
    success {
      echo 'CI/CD Pipeline completed successfully'
    }

    failure {
      echo 'CI/CD Pipeline failed'
    }
  }
}
```

← → ↻ ⚠ Not secure 54.172.207.1:8080/job/Hiring-App-Declarative/configure ☆ r ⋮

 **Jenkins** / Hiring-App-Declarative ▾ / Configure 🔍 ⚙️ 👤

Pipeline script

Script ?

```
5 stages {
6
7     stage('Git Clone') {
8         steps {
9             git branch: 'main',
10                url: 'https://github.com/rajkumari-hub/hiring-app'
11         }
12     }
13
14     stage('Build') {
15         steps {
16             sh 'mvn clean compile'
17         }
18     }
19 }
```

☒ Use Groovy Sandbox ?

[Pipeline Syntax](#)

Advanced

What happens?

pipeline



agent



Git Clone



Build



Test



Package

pipeline {} is the main Declarative Pipeline block. agent any tells Jenkins to allocate an available executor, while stages contains the actual work.

Click:

Save

Then: Build Now

The screenshot shows the Jenkins web interface for a pipeline named 'Hiring-App-Declarative'. The left sidebar contains navigation links: Status (selected), Changes, Build Now, Delete Pipeline, Configure, Rename, Stages, and Pipeline Syntax. The main area displays the pipeline status as 'Hiring-App-Declarative' with a green checkmark. Below this, there are 'Permalinks' for various build events. At the bottom left, a 'Builds' section shows a list of builds: #2 (1:52 PM, green) and #1 (1:48 PM, red). The bottom part of the image shows the console output for build #2, which is successful. The console output includes information about Maven plugins, packaging, and the final success message.

← → ↻ Not secure 54.172.207.1:8080/job/Hiring-App-Declarative/

Jenkins / Hiring-App-Declarative

Status

Changes

Build Now

Delete Pipeline

Configure

Rename

Stages

Pipeline Syntax

Hiring-App-Declarative

Permalinks

- Last build (#2), 1 min 50 sec ago
- Last stable build (#2), 1 min 50 sec ago
- Last successful build (#2), 1 min 50 sec ago
- Last failed build (#1), 5 min 15 sec ago
- Last unsuccessful build (#1), 5 min 15 sec ago
- Last completed build (#2), 1 min 50 sec ago

Builds > Filter

Today

- #2 1:52 PM
- #1 1:48 PM

← → ↻ Not secure 54.172.207.1:8080/job/Hiring-App-Declarative/2/console

Jenkins / Hiring-App-Declarative / #2

```
[INFO] --- maven-surefire-plugin:2.12.4:test (default-test) @ hiring ---
[INFO] Tests are skipped.
[INFO] --- maven-war-plugin:3.3.1:war (default-war) @ hiring ---
[INFO] Packaging webapp
[INFO] Assembling webapp [hiring] in [/var/lib/jenkins/workspace/Hiring-App-Declarative/target/hiring]
[INFO] Processing war project
[INFO] Copying webapp resources [/var/lib/jenkins/workspace/Hiring-App-Declarative/src/main/webapp]
[INFO] Building war: /var/lib/jenkins/workspace/Hiring-App-Declarative/target/hiring.war
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.854 s
[INFO] Finished at: 2026-08-17T13:52:19Z
[INFO] -----
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
CI/CD Pipeline completed successfully
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Part 3 — Create Scripted Pipeline Job

Now create your second job.

Step 1

Go to:

Dashboard



New Item

Job name:

Hiring-App-Scripted

Select:

Pipeline

Click:

OK

←

→

↻

⚠ Not secure

54.172.207.1:8080/view/all/newJob

☆

Ⓐ

⋮

 **Jenkins**

/ All ▾ / New Item

🔍

⚙

👤

New Item

Enter an item name

Hiring-App-Scripted

Select an item type

 **Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

 **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

 **Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

 **Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

OK

Step 2: Pipeline script

```

node {

    stage('Git Clone') {
        git branch: 'main',
            url: 'https://github.com/betawins/hiring-app.git'
    }

    stage('Build') {
        sh 'mvn clean compile'
    }

    stage('Test') {
        sh 'mvn test'
    }

    stage('Package') {
        sh 'mvn package -DskipTests'
    }

    echo 'Scripted Pipeline completed successfully'
}

```

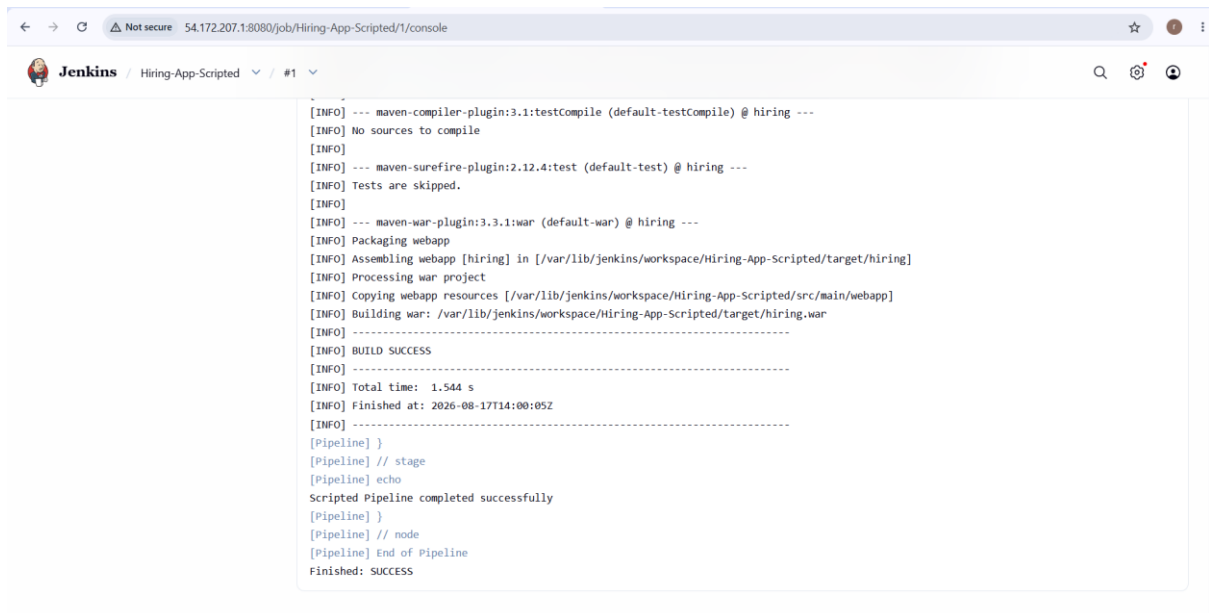
Click:

Save

Then:

Build Now

The screenshot shows the Jenkins web interface for a pipeline named 'Hiring-App-Scripted'. The browser address bar shows the URL '54.172.207.1:8080/job/Hiring-App-Scripted/'. The Jenkins logo and name are at the top left. A sidebar on the left contains links: Status (selected), Changes, Build Now, Delete Pipeline, Configure, Rename, Stages, and Pipeline Syntax. The main content area shows the pipeline status as 'Hiring-App-Scripted' with a green checkmark icon. Below this, there is a 'Permalinks' section with four links: 'Last build (#1), 54 sec ago', 'Last stable build (#1), 54 sec ago', 'Last successful build (#1), 54 sec ago', and 'Last completed build (#1), 54 sec ago'. At the bottom, a 'Builds' panel shows a search filter and a list of builds: '#2 2:00 PM' and '#1 1:59 PM', both with green status icons and dropdown arrows.



The screenshot shows a web browser window displaying the Jenkins console output for a job named 'Hiring-App-Scripted'. The output is a log of Maven build steps, including compilation, testing, and packaging. The build is successful, as indicated by the 'BUILD SUCCESS' message. The console output is as follows:

```
[INFO] --- maven-compiler-plugin:3.1:testCompile (default-testCompile) @ hiring ---
[INFO] No sources to compile
[INFO]
[INFO] --- maven-surefire-plugin:2.12.4:test (default-test) @ hiring ---
[INFO] Tests are skipped.
[INFO]
[INFO] --- maven-war-plugin:3.3.1:war (default-war) @ hiring ---
[INFO] Packaging webapp
[INFO] Assembling webapp [hiring] in [/var/lib/jenkins/workspace/Hiring-App-Scripted/target/hiring]
[INFO] Processing war project
[INFO] Copying webapp resources [/var/lib/jenkins/workspace/Hiring-App-Scripted/src/main/webapp]
[INFO] Building war: /var/lib/jenkins/workspace/Hiring-App-Scripted/target/hiring.war
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.544 s
[INFO] Finished at: 2026-08-17T14:00:05Z
[INFO] -----
[Pipeline] }
[Pipeline] // stage
[Pipeline] echo
Scripted Pipeline completed successfully
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Important difference

Declarative:

pipeline {

agent any

stages {

stage('Build') {

steps {

sh 'mvn clean compile'

}

}

}

}

Scripted:

node {

stage('Build') {

```
        sh 'mvn clean compile'
    }

}
```

Scripted Pipeline is essentially Groovy-based and gives you much more programming flexibility. Jenkins describes Declarative as more structured/opinionated, while Scripted is more flexible and imperative.

Part 4 — Create Multi-Stage Pipeline Job

Your third assignment is the **multi-stage pipeline**.

Here, we will make the CI/CD flow very clear:

Stage 1 → Checkout

↓

Stage 2 → Build

↓

Stage 3 → Test

↓

Stage 4 → Package

↓

Stage 5 → Deploy

Step 1

Create:

New Item

Job name:

Hiring-App-Multi-Stage

Select:

Pipeline

Click:

OK

← → ↻ ⚠ Not secure 54.172.207.1:8080/view/all/newJob ☆ ⚙

Jenkins / All ▾ / New Item 🔍 ⚙ 👤

New Item

Enter an item name

Hiring-App-Multi-Stage

Select an item type

- Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
- Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

Step 2: Multi-stage the Pipeline

```

pipeline {
    agent any

    stages {

        stage('Checkout') {
            steps {
                echo 'Checking out source code...'

                git branch: 'main',
                    url: 'https://github.com/betawins/hiring-app.git'
            }
        }

        stage('Build') {
            steps {
                echo 'Building application...'

                sh 'mvn clean compile'
            }
        }

        stage('Test') {
            steps {
                echo 'Running tests...'

                sh 'mvn test'
            }
        }

        stage('Package') {
            steps {
                echo 'Creating application package...'

                sh 'mvn package -DskipTests'
            }
        }

        stage('Deploy') {
            steps {
                echo 'Deployment stage completed'
                echo 'Ready to deploy application to Tomcat'
            }
        }
    }

    post {
        success {
            echo 'Multi-stage CI/CD pipeline completed successfully'
        }

        failure {
            echo 'Multi-stage CI/CD pipeline failed'
        }
    }
}

```

Click:

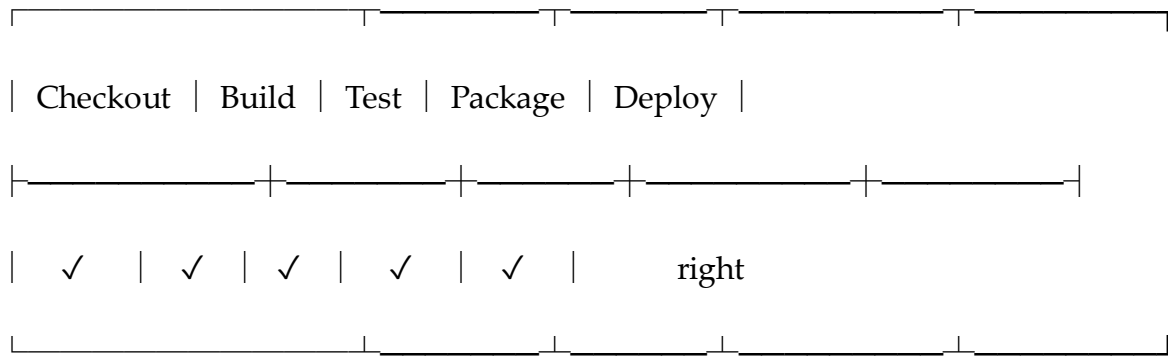
Save

Then:

Build Now

What you should see in Jenkins

Jenkins Stage View should show something similar to:



This demonstrates that you understand **multiple stages in a CI/CD pipeline**. Jenkins uses stage blocks to represent logically distinct portions of the delivery process.

← → ↺ ⚠ Not secure 54.172.207.1:8080/job/Hiring-App-Multi-Stage/configure ☆ ⓘ ⋮

 **Jenkins** / Hiring-App-Multi-Stage ▾ / Configure

🔍 ⚙️ 👤

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script ▾

Script ?

try sample Pipeline... ▾

```
1 pipeline {
2     agent any
3
4     stages {
5
6         stage('Checkout') {
7             steps {
8                 echo 'Checking out source code...'
9
10                git branch: 'main',
11                  url: 'https://github.com/rajkumari-hub/hiring-app'
12            }
13        }
14
15        stage('Build') {
```

☒ Use Groovy Sandbox ?

Save

Apply

← → ↺ ⚠ Not secure 54.172.207.1:8080/job/Hiring-App-Multi-Stage/ ☆ ⓘ ⋮

 **Jenkins** / Hiring-App-Multi-Stage

🔍 ⚙️ 👤

Status

</> Changes

▶ Build Now

🗑 Delete Pipeline

⚙ Configure

✎ Rename

📅 Stages

🔗 Pipeline Syntax

Hiring-App-Multi-Stage

Permalinks

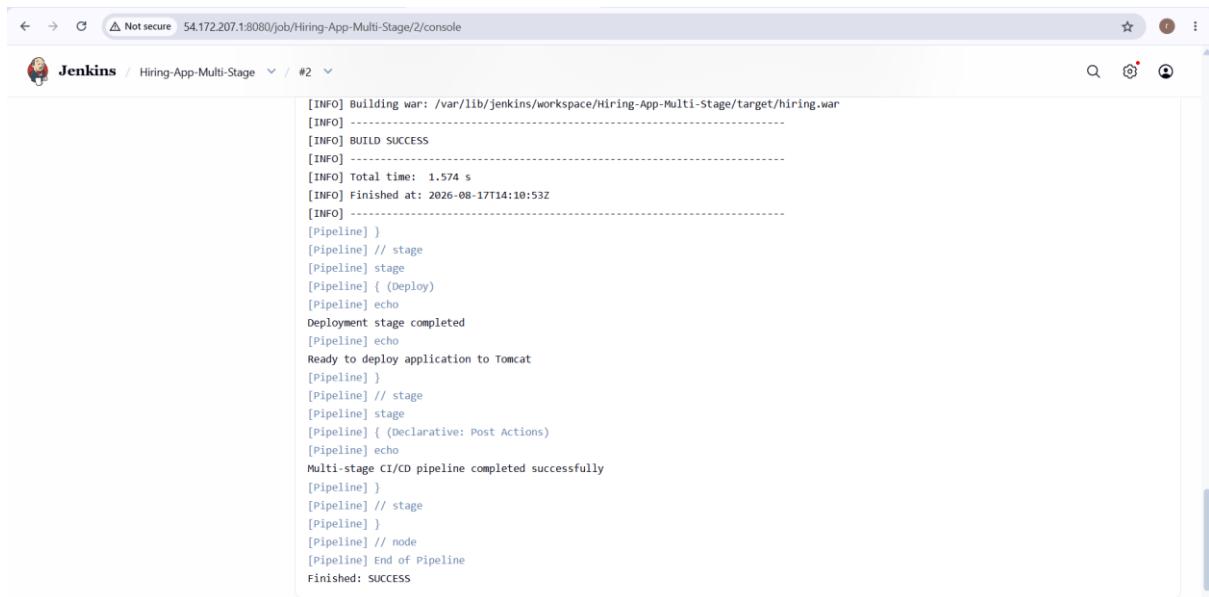
Add description

Builds > ⋮

Today

🟢 #2 2:10 PM ▾

🟢 #1 2:10 PM ▾



The screenshot shows the Jenkins web interface with a console log for a job named 'Hiring-App-Multi-Stage'. The log displays the following information:

```
[INFO] building war: /var/lib/jenkins/workspace/Hiring-App-Multi-Stage/target/hiring.war
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.574 s
[INFO] Finished at: 2026-08-17T14:10:53Z
[INFO] -----
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deployment stage completed
[Pipeline] echo
Ready to deploy application to Tomcat
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Multi-stage CI/CD pipeline completed successfully
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Part 5 — Create Parallel Stage Pipeline

This is your fourth assignment.

The important concept is:

Normal stages

Build

↓

Test 1

↓

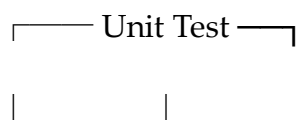
Test 2

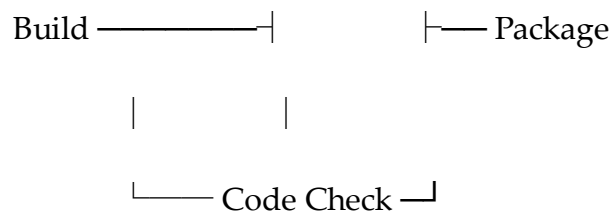
↓

Package

Everything waits for the previous stage.

Parallel stages





Unit Test and Code Check can execute at the same time.

Jenkins Declarative Pipeline supports a parallel section containing nested stages.

Step 1

Create:

New Item

Job name:

Hiring-App-Parallel

Select:

Pipeline

Click:

OK

← → ↺

Not secure 54.172.207.1:8080/view/all/newJob

☆

⌵

⋮

 **Jenkins** / All ▾ / New Item

🔍 ⚙️ 👤

New Item

Enter an item name

Hiring-App-Parallel

Select an item type

 **Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

 **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

 **Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

 **Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

OK

Step 2: Parallel Pipeline

```
pipeline {
    agent any

    stages {

        stage('Checkout') {
            steps {
                echo 'Checking out source code...'

                git branch: 'main',
                    url: 'https://github.com/betawins/hiring-app.git'
            }
        }

        stage('Build') {
            steps {
                echo 'Building application...'

                sh 'mvn clean compile'
            }
        }

        stage('Parallel Testing') {

            parallel {

                stage('Unit Test') {
                    steps {
                        echo 'Running Unit Tests...'

                        sh 'mvn test'
                    }
                }
            }
        }
    }
}
```

```

        sh 'mvn test'
    }
}

stage('Code Check') {
    steps {
        echo 'Performing Code Check...'

        sh 'mvn validate'
    }
}

}

stage('Package') {
    steps {
        echo 'Creating application package...'

        sh 'mvn package -DskipTests'
    }
}

}

post {
    success {
        echo 'Parallel CI/CD pipeline completed successfully'
    }

    failure {
        echo 'Parallel CI/CD pipeline failed'
    }
}
}

```

← → ↻ ⚠ Not secure 54.172.207.1:8080/job/Hiring-App-Parallel/configure ☆ 🧑

Jenkins / Hiring-App-Parallel ▾ / Configure 🔍 ⚙️ 🧑

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script ▾

Script ?

```
1 pipeline {
2     agent any
3
4     stages {
5
6         stage('Checkout') {
7             steps {
8                 echo 'Checking out source code...'
9
10                git branch: 'main',
11                   url: 'https://github.com/rajkumari-hub/hiring-app'
12            }
13        }
14
15        stage('Build') {
```

try sample Pipeline... ▾

☒ Use Groovy Sandbox ?

Save Apply

Click:

Save

Then:

Build Now

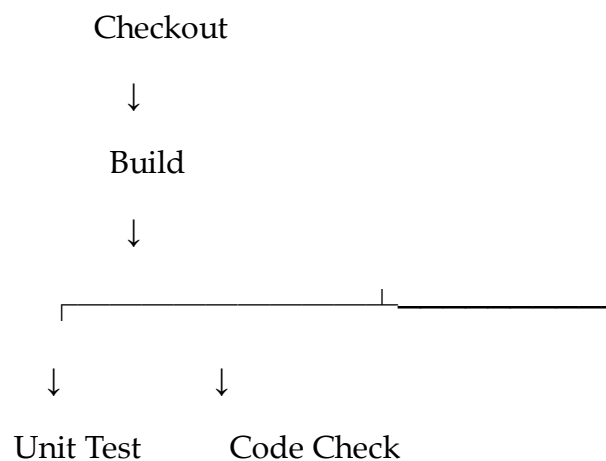
The screenshot shows the Jenkins web interface for a pipeline named "Hiring-App-Parallel". The top navigation bar includes "Status", "Changes", "Build Now", "Delete Pipeline", "Configure", "Rename", "Stages", and "Pipeline Syntax". The "Status" tab is active, showing a green checkmark and the pipeline name. Below the navigation bar, there are "Permalinks" for the last build, last stable build, last successful build, and last completed build, all indicating they are 28 seconds ago. On the left, a "Builds" section shows a list of builds: #4, #3, #2, and #1, all with green checkmarks and timestamps of 2:30 PM. The right side of the page is mostly blank, suggesting a description or additional details that are not visible.

The screenshot shows the Jenkins console output for the "Hiring-App-Parallel" pipeline, build #3. The output is a log of the pipeline's execution, showing various steps and their results. The log includes information about the Maven Surefire plugin, Maven War plugin, and the packaging of the webapp. It also shows the assembly of the webapp, copying of resources, and the building of the war file. The final output is "BUILD SUCCESS" and "Parallel CI/CD pipeline completed successfully".

```
[INFO] --- maven-surefire-plugin:2.12.4:test (default-test) @ hiring ---
[INFO] Tests are skipped.
[INFO] --- maven-war-plugin:3.3.1:war (default-war) @ hiring ---
[INFO] Packaging webapp
[INFO] Assembling webapp [hiring] in [/var/lib/jenkins/workspace/Hiring-App-Parallel/target/hiring]
[INFO] Processing war project
[INFO] Copying webapp resources [/var/lib/jenkins/workspace/Hiring-App-Parallel/src/main/webapp]
[INFO] Building war: /var/lib/jenkins/workspace/Hiring-App-Parallel/target/hiring.war
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.173 s
[INFO] Finished at: 2026-08-17T14:31:19Z
[INFO] -----
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Parallel CI/CD pipeline completed successfully
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Part 6 — Understand the Parallel Pipeline

The execution will look like this:





Package

The important part is:

```
stage('Parallel Testing') {
```

```
    parallel {
```

```
        stage('Unit Test') {
```

```
            steps {
```

```
                sh 'mvn test'
```

```
            }
```

```
        }
```

```
        stage('Code Check') {
```

```
            steps {
```

```
                sh 'mvn validate'
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

Jenkins starts those nested stages concurrently, subject to available executors/agents.

Part 7 — Verify All Four Jobs

After creating them, your Jenkins Dashboard should have:

Hiring-App-Declarative

Hiring-App-Scripted

Hiring-App-Multi-Stage

Hiring-App-Parallel

Run each one.

Job 1

Hiring-App-Declarative

Expected:

SUCCESS

Stages:

Git Clone

Build

Test

Package

Job 2

Hiring-App-Scripted

Expected:

SUCCESS

Stages:

Git Clone

Build

Test

Package

Job 3

Hiring-App-Multi-Stage

Expected:

SUCCESS

Stages:

Checkout

Build

Test

Package

Deploy

Job 4

Hiring-App-Parallel

Expected:

SUCCESS

Stages:

Checkout

Build

└── Unit Test

|

└── Code Check

↓

Package

Part 8 – What to explain to your trainer

You can explain the assignment like this:

1. Declarative Pipeline

"Declarative Pipeline uses a predefined and structured syntax. It uses the pipeline, agent, stages, stage, and steps blocks. I used it to implement Git checkout, Maven build, testing and packaging."

2. Scripted Pipeline

"Scripted Pipeline is Groovy-based and provides more programming flexibility. Instead of the Declarative pipeline block, I used a node block and defined each stage using Groovy syntax."

3. Multi-Stage Pipeline

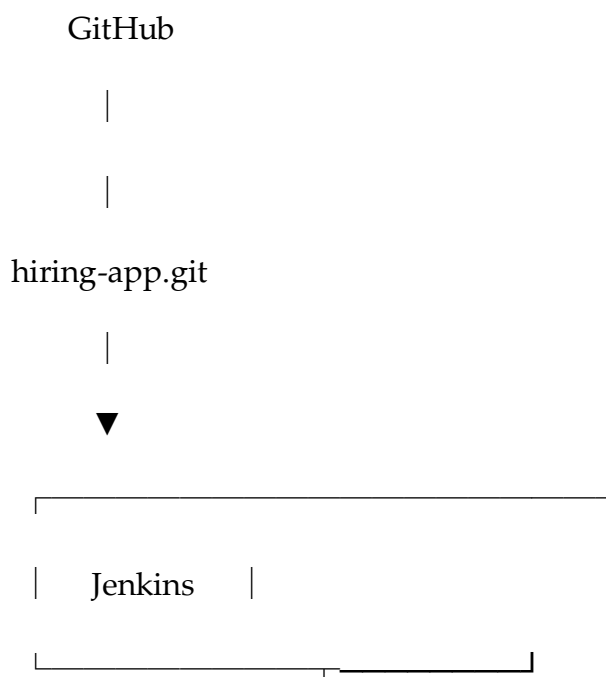
"The multi-stage pipeline divides the CI/CD process into independent stages such as Checkout, Build, Test, Package and Deploy. The stages execute sequentially."

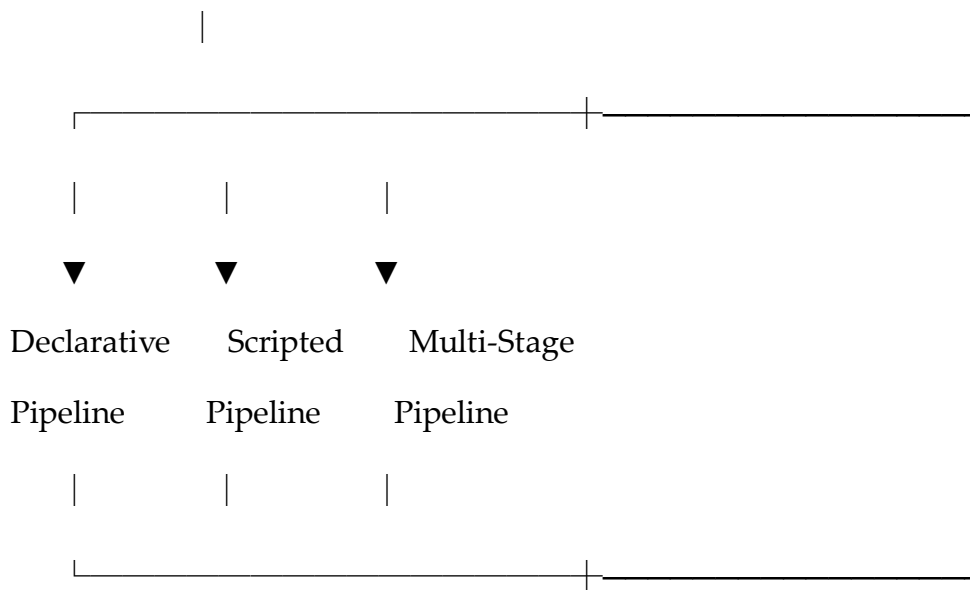
4. Parallel Pipeline

"The parallel pipeline executes independent tasks simultaneously. In my implementation, Unit Test and Code Check run in parallel after Build, and Package runs after both parallel branches complete."

Final Assignment Architecture

Your complete assignment can be shown as:





▼
Build

▼
Test

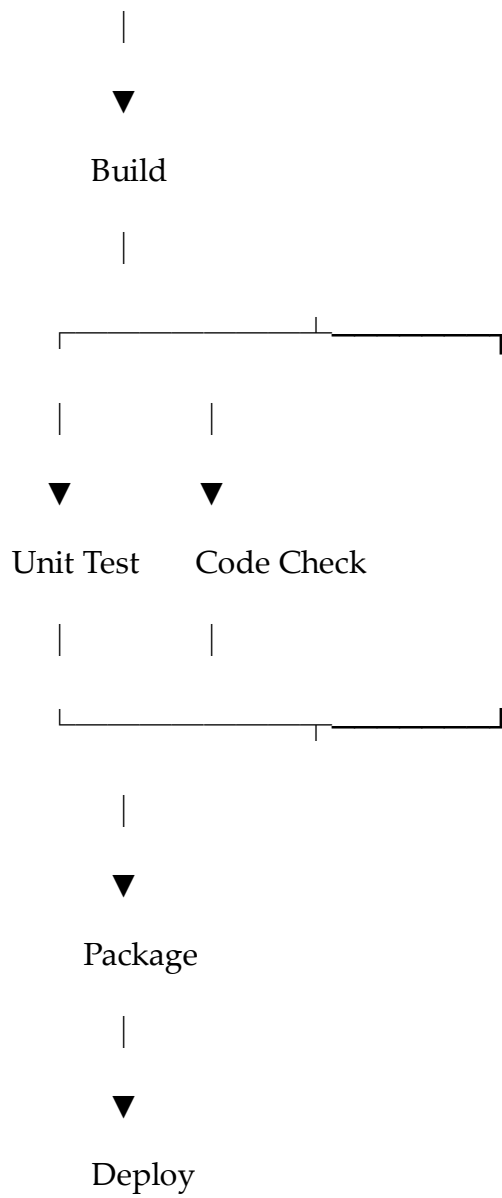
▼
Package

▼
Deploy

And the parallel version:

GitHub

▼
Checkout





Jenkins

/ Hiring-App-Scripted ▾ / Configure



Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script ▾

Script ?

```
3 stage('Git Clone') {
4     git branch: 'main',
5     url: 'https://github.com/rajkumari-hub/hiring-app'
6 }
7
8 stage('Build') {
9     sh 'mvn clean compile'
10 }
11
12 stage('Test') {
13     sh 'mvn test'
14 }
15
16 stage('Package') {
17     sh 'mvn package -DskipTests'
```

try sample Pipeline... ▾

☒ Use Groovy Sandbox ?

Save

Apply